

You should be careful about running commands that you don't fully understand. However, if you have good reason to believe that a command can do little harm, and if you trust the source of the recommendation, it might be reasonable to try the suggestions offered by the tools you're using.

NOTE

Keep in mind there are individuals who will tell you to run commands that will wipe your system or expose your system to remote exploitation. Following the suggestions of a tool provided by a company or organization you trust is different from following the suggestions of random people online. Anytime you're dealing with remote connections, proceed with an abundance of caution.

Read the Log Output

As mentioned earlier, the log output that you see when you run a command like `platform push` can be both informative and intimidating. Read through the following snippet of log output, taken from a different attempt at using `platform push`, and see if you can spot the issue:

```
--snip--
Collecting soupsieve==2.3.2.post1
  Using cached soupsieve-2.3.2.post1-py3-none-any.whl (37 kB)
Collecting sqlparse==0.4.2
  Using cached sqlparse-0.4.2-py3-none-any.whl (42 kB)
Installing collected packages: platformshconfig, sqlparse,...
Successfully installed Django-4.1 asgiref-3.5.2 beautifulsoup4-4.11.1...
W: ERROR: Could not find a version that satisfies the requirement gunicornn
W: ERROR: No matching distribution found for gunicornn

130 static files copied to '/app/static'.

Executing pre-flight checks...
--snip--
```

When a deployment attempt fails, a good strategy is to look through the log output and see if you can spot anything that looks like warnings or errors. Warnings are fairly common; they're often messages about upcoming changes in a project's dependencies, to help developers address issues before they cause actual failures.

A successful push may have warnings, but it shouldn't have any errors. In this case, Platform.sh couldn't find a way to install the requirement `gunicornn`. This is a typo in the `requirements_remote.txt` file, which was supposed to include `gunicorn` (with one *r*). It's not always easy to spot the root issue in log output, especially when the problem causes a bunch of cascading errors and warnings. Just like when reading a traceback on your local system, it's a good idea to look closely at the first few errors that are listed, and also the last few errors. Most of the errors in between tend to be internal packages complaining that something went wrong, and passing messages about the error to other internal packages. The actual error we can fix is usually one of the first or last errors listed.